Hunting for Bits in the Shadow of Regulation: Entropy-Based Characterization of Compiler-Induced Side-Channel Leaks

Paul Fuchs, Kerstin Lemke-Rust, and Stefan Schiffner

Institute for Cyber Security & Privacy
Bonn-Rhein-Sieg University of Applied Sciences
53757 Sankt Augustin, Germany
name.surname@h-brs.de

Abstract. Side-channel security has evolved rapidly over the decades and is increasingly becoming a central concern for developers of cryptographic implementations on general-purpose systems. While the Constant-Time (CT) coding paradigm is establishing itself as a standard practice to mitigate timing-based side-channel leakage, recent publications [GPS26; Sch+25] demonstrate that compiler-induced CT violations are widespread and can impact even hardened cryptographic implementations. These optimizations can silently transform secure source code into vulnerable binary executables.
However, a critical question remains: To what extent does this compiler-induced leakage degrade the security margin of a cryptographic primitive in the field? To the best of our knowledge, no existing tooling is available to quantify this phenomenon. This research aims to fill this gap by providing an entropy-based quantification of the timing leakage magnitude. By establishing a measurable scale of vulnerability, this work seeks to evaluate whether the industry should adopt regulatory-grade practices—such as the reproducibility requirements found in FIPS 140-3 and ISO/IEC 19790—for open-source cryptographic libraries. If the quantified impact is shown to be significant, adopting these standards could ensure that the binary artifact deployed by the end-user is functionally equivalent to the version rigorously tested by the developer.

Keywords: Side-Channel Analysis · Leakage Assessment · Leakage Quantification · Mutual Information Analysis · Conditional Entropy · Deep Learning

1 Introduction

Timing attacks, first introduced by Paul Kocher in 1996 [Koc96], fundamentally altered the security landscape by enabling attackers to recover sensitive key material through the analysis of execution time variances. As these vulnerabilities became more sophisticated, the focus shifted toward proactive Leakage Assessment by statistically evaluating execution times, under fixed vs. randomly-selected secret input, commonly using Welch's t-test, to identify secret-dependent

timing behaviour, implying the potential of timing side channel attacks [GJR+11; Bec+13; SM15; RBV17]. A variety of CT Analysis tools have been established over the time, spanning the fields of *statistical testing* [RBV17; CSC24], [Dun+25], *dynamical testing* [Lan10; Wic+18; Wei+18], and *static testing* [Alm+16; CSC24; DBR20] methods. These tools enable systematically testing of cryptographic implementations on binary- or intermediate level, and allow scalable analysis, which was conducted most recently to uncover compiler-induced CT violations [GPS26; Sch+25]. Constant time compilers, e.g. [Alm+17], did not establish for practical reasons, such as language or dependencies.

Although exemplary testing methods itself might be used to somehow rank the vulnerability, e.g. comparing t-values or amount of findings returned by dynamic binary testing, it does not answer how many bits of the secret key are information theoretically disclosed by the timing side channel. Therefore, security metrics have been established, such as *Guessing Entropy*, *Success Rate*, [SMY09], *Cross Entropy Ratio* [Zha+20], or *(Effective) Perceived information* and *Latent Perceived Information* [IUH24].

2 Methodology

2.1 Dataset Construction and Ground Truth Establishment

To evaluate the impact of compiler optimizations, we construct a dataset with compiler-induced CT violations. First, we select representative, well-researched CT Source Code implementations. These are compiled using a range of optimization levels (-O0,...,-O3, -Ofast, -Os), to cover the diverse build situation found in unregularized software ecosystems. We employ automated CT-analysis tools to identify potential violations, which are subsequently validated via manual disassembly inspection. This process establishes a *ground truth* regarding the cryptographic impact of compiler-induced timing bahaviour.

2.2 Side-Channel Trace Acquisition

We adopt a leakage-model approach to quantify security loss. We execute the identified binaries on x86-64 architectures and capture micro-architectural traces, focusing on timing variations and cache-access patterns. The resulting dataset $\mathcal{D}$ consists of pairs of secret vectors and their corresponding execution traces: $\mathcal{D} = \{(secret_i, trace_i)\}_{i=1}^{N}$.

2.3 Entropy-Based Leakage Characterization

To quantify the information leakage, we utilize Deep Learning (DL) for automated feature extraction from high-dimensional traces. Following the principles of [IUH24], our approach estimates the perceived information, specifically the amount of entropy reduction regarding the secret provided by the observed traces. This allows us to characterize the severity of compiler-induced leaks in a mathematically rigorous, entropy-based framework.

3 Results and Discussion

The primary contribution of this work is a practical framework for the quantitative assessment of timing side-channels on x86-64. This approach directly addresses the research question of how severe compiler-induced leaks are by providing a measurable metric: entropy reduction.

Independent Observations show that compilers like Clang can even circumvent established side-channel countermeasures [1]. This instance, encountered during the preliminary stages of this research, serves as an exemplar illustrating the urgent need for a systematic and large-scale automated investigation into compiler-induced vulnerabilities. While regulatory frameworks such as FIPS 140-3 or ISO/IEC 19790 effectively neutralize such leaks by mandating strict reproducibility between the evaluated and deployed targets, this phenomenon remains a critical blind spot in unregulated software ecosystems.

While this work serves as a proof-of-concept, we identify key limitations: the current implementation is tailored for x86-64 and the computational complexity of deep-learning-based extraction poses challenges for large-scale automation.

4 Conclusions

In conclusion, this work moves the field from qualitative detection of leaks to a practical quantification of security loss. By providing an entropy-based scale, we enable to evaluate whether regulatory-grade practices, such as FIPS 140-3, are required to ensure binary-level security. Future work will focus on scaling this framework to broader hardware architectures and integrating it into automated build pipelines.

[1] `https://github.com/randombit/botan/blob/ed0ced67c852c6/src/ct_selftest/ct_selftest.cpp#L144`

References

[Alm+16] José Bacelar Almeida et al. "Verifying Constant-Time Implementations". Ed. by Thorsten Holz and Stefan Savage. USENIX Association, 2016. URL: https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/almeida.

[Alm+17] José B. Almeida et al. "Jasmin: High-assurance and high-speed cryptography". ACM, 2017. DOI: 10.1145/3133956.3134078.

[Bec+13] George Becker et al. "Test vector leakage assessment (TVLA) methodology in practice". sn. 2013.

[CSC24] Luwei Cai, Fu Song, and Taolue Chen. "Towards Efficient Verification of Constant-Time Cryptographic Implementations". *Proceedings of the ACM on Software Engineering* (2024). DOI: 10.1145/3643772.

[DBR20] Lesly-Ann Daniel, Sebastien Bardin, and Tamara Rezk. "Binsec/Rel: Efficient Relational Symbolic Execution for Constant-Time at Binary-Level". IEEE, 2020. DOI: 10.1109/sp40000.2020.00074.

[Dun+25] Martin Dunsche et al. *SILENT: A New Lens on Statistics in Software Timing Side Channels*. 2025. DOI: 10.48550/ARXIV.2504.19821.

[GJR+11] Benjamin Jun Gilbert Goodwill, Josh Jaffe, Pankaj Rohatgi, et al. "A testing methodology for side-channel resistance validation". 2011.

[GPS26] Lukas Gerlach, Robert Pietsch, and Michael Schwarz. "Do Compilers Break Constant-Time Guarantees?" Springer Nature Switzerland, 2026. DOI: 10.1007/978-3-032-07035-7_20.

[IUH24] Akira Ito, Rei Ueno, and Naofumi Homma. *Perceived Information Revisited II: Information-Theoretical Analysis of Deep-Learning Based Side-Channel Attacks*. 2024. URL: https://eprint.iacr.org/2024/124.

[Koc96] Paul C. Kocher. "Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems". Ed. by Neal Koblitz. Springer, 1996. DOI: 10.1007/3-540-68697-5_9.

[Lan10] Adam Langley. *Checking that functions are constant time with Valgrind*. 2010. URL: https://github.com/agl/ctgrind.

[RBV17] Oscar Reparaz, Josep Balasch, and Ingrid Verbauwhede. "Dude, is my code constant time?" IEEE, 2017. DOI: 10.23919/date.2017.7927267.

[Sch+25] Moritz Schneider et al. “Breaking Bad: How Compilers Break Constant-Time Implementations”. ACM, 2025. DOI: 10.1145/3708821.3733909.

[SM15] Tobias Schneider and Amir Moradi. “Leakage Assessment Methodology: A Clear Roadmap for Side-Channel Evaluations”. Springer Berlin Heidelberg, 2015. DOI: 10.1007/978-3-662-48324-4_25.

[SMY09] François-Xavier Standaert, Tal G. Malkin, and Moti Yung. “A Unified Framework for the Analysis of Side-Channel Key Recovery Attacks”. Springer Berlin Heidelberg, 2009. DOI: 10.1007/978-3-642-01001-9_26.

[Wei+18] Samuel Weiser et al. “DATA - Differential Address Trace Analysis: Finding Address-based Side-Channels in Binaries”. Ed. by William Enck and Adrienne Porter Felt. USENIX Association, 2018. URL: https://www.usenix.org/conference/usenixsecurity18/presentation/weiser.

[Wic+18] Jan Wichelmann et al. “MicroWalk: A Framework for Finding Side Channels in Binaries”. ACM, 2018. DOI: 10.1145/3274694.3274741.

[Zha+20] Jiajia Zhang et al. “A Novel Evaluation Metric for Deep Learning-Based Side Channel Analysis and Its Extended Application to Imbalanced Data”. *IACR Transactions on Cryptographic Hardware and Embedded Systems* (2020). DOI: 10.46586/tches.v2020.i3.73-96.